

# Metropolis Algorithm

## Chapter 4 — MCMC from Scratch

Sharath Kumar Yerramada (BSD-BG-2417)   Peravena Yashwanth  
(BSD-BG-2411)   Rathod Akash (BSD-BG-2415)   Dipendu Halder  
(BSD-BG-2404)

Indian Statistical Institute

May 6, 2026

# What is MCMC?

## The Problem:

- We want to compute:  $\langle f(x) \rangle = \int f(x) P(x) dx$
- For complex distributions this integral is **analytically intractable**
- Direct sampling from  $P(x)$  is often **impossible**

## The MCMC Solution:

- Generate a sequence  $x^{(0)}, x^{(1)}, x^{(2)}, \dots$  (a Markov chain)
- The chain converges to the target distribution  $P(x)$
- Estimate:  $\langle f(x) \rangle \approx \frac{1}{K} \sum_{k=1}^K f(x^{(k)})$

## Key Point

**Metropolis Algorithm** is the most famous version of MCMC

# The Target Distribution

Suppose the probability distribution  $P(x)$  is written as:

$$P(x) = \frac{e^{-S(x)}}{Z}$$

- $S(x)$  — called the **action** (negative log-likelihood in statistics)
- $Z = \int e^{-S(x)} dx$  — the **partition function** (normalising constant)

## Key observations:

- In practice,  $S(x)$  is **known**
- $Z$  is **unknown** — computing it requires the very integral we want to avoid
- For Gaussian:  $S(x) = \frac{x^2}{2}$ ,  $P(x) = \frac{e^{-x^2/2}}{\sqrt{2\pi}}$

## Critical Insight

$Z$  cancels in the acceptance step — we **never** need to compute it

# The Metropolis Algorithm

Starting from initial value  $x^{(0)}$ , at each step  $k$ :

- 1 **Propose:** Draw  $\Delta x \sim \text{Uniform}(-c, +c)$ , set  $x' = x^{(k)} + \Delta x$
- 2 **Compute:**  $\Delta S = S(x') - S(x^{(k)})$
- 3 **Accept**  $x'$  with probability  $\min(1, e^{-\Delta S})$
- 4 **Update:**

$$x^{(k+1)} = \begin{cases} x' & \text{if accepted} \\ x^{(k)} & \text{if rejected} \end{cases}$$

## Step size $c$

$c > 0$  is a **free tuning parameter** we choose. It controls how far each proposal jumps.

## Why accept worse moves?

To prevent the chain getting stuck in local regions — occasional downhill moves allow full exploration of  $P(x)$

# The Metropolis Test

## Practical implementation:

- Draw  $r \sim \text{Uniform}(0, 1)$
- Accept  $x'$  if  $r < e^{S(x^{(k)}) - S(x')}$
- Otherwise reject and keep  $x^{(k+1)} = x^{(k)}$

## Intuition

Higher probability point  $\rightarrow$  always accept

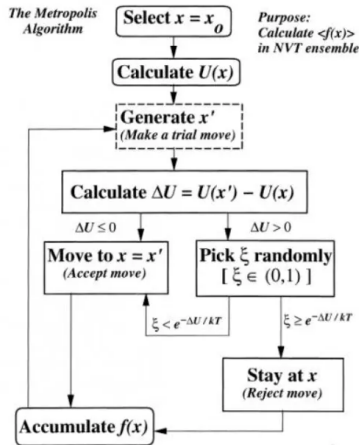
Lower probability point  $\rightarrow$  accept with some chance, not zero

## Why this is enough

We only need to **evaluate**  $S(x)$  at any point

We never need to **integrate**  $e^{-S(x)}$  to find  $Z$

# The Original Metropolis Flowchart



- Published in 1953 by Metropolis, Rosenbluth and Teller
- $U(x)$  is their notation for  $S(x)$  — the energy/action
- $\xi$  is their notation for  $r$  — the uniform random number

## Bottom line

Different notation, identical idea — this is all Metropolis is

# Why $Z$ Cancels — The Key Idea

The acceptance probability is:

$$\begin{aligned}\min\left(1, e^{S(x^{(k)})-S(x')}\right) &= \min\left(1, \frac{e^{-S(x')}}{e^{-S(x^{(k)})}}\right) = \min\left(1, \frac{P(x') \cdot Z}{P(x^{(k)}) \cdot Z}\right) \\ &= \min\left(1, \frac{P(x')}{P(x^{(k)})}\right)\end{aligned}$$

$Z$  appears in both numerator and denominator  
 $\Rightarrow$  it cancels exactly

**This is the entire power of the Metropolis algorithm:**

- We only need the **ratio** of probabilities
- The ratio requires only  $S(x)$  — which we know
- We never need  $Z$  — which we do not know

# Gaussian Example — Concrete Numbers

**Target:**  $P(x) = \frac{e^{-x^2/2}}{\sqrt{2\pi}}$

**In Metropolis form:**

- $S(x) = \frac{x^2}{2}$  (we know this)
- $Z = \sqrt{2\pi}$  (we do NOT need this)

**Acceptance probability for proposal  $x \rightarrow x'$ :**

$$e^{S(x)-S(x')} = e^{\frac{x^2}{2} - \frac{x'^2}{2}}$$

**Interpretation:**

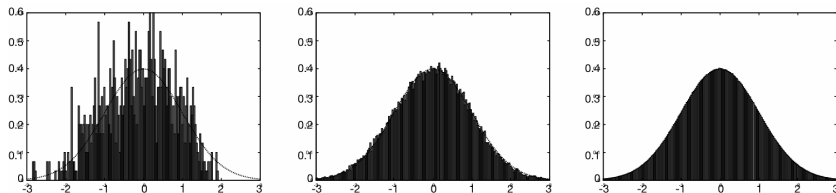
- If  $|x'| < |x|$ : moving closer to centre  $\Rightarrow e^{(\cdot)} > 1 \Rightarrow$  always accept
- If  $|x'| > |x|$ : moving away from centre  $\Rightarrow$  accept with probability  $< 1$

**Initial conditions used in the textbook**

$$x^{(0)} = 0, \quad \text{step size } c = 0.5$$



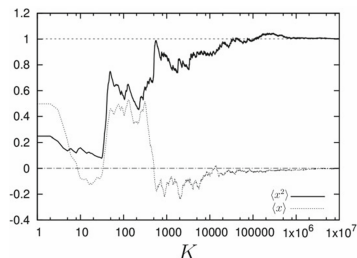
# Histogram Convergence — Fig. 4.1



**Fig. 4.1** The histogram of  $x^{(1)}, x^{(2)}, \dots, x^{(K)}$  for  $K = 10^3, 10^5$  and  $10^7$ . It converges to the target distribution  $P(x) = \frac{e^{-x^2/2}}{\sqrt{2\pi}}$  as  $K$  becomes large

- Histograms shown for  $K = 10^3, 10^5, 10^7$  samples
- As  $K$  increases, histogram converges to  $P(x) = \frac{e^{-x^2/2}}{\sqrt{2\pi}}$
- More samples  $\Rightarrow$  smoother and more accurate

## Expectation Values Converging — Fig. 4.2



**Fig. 4.2**  $\langle x \rangle = \frac{1}{K} \sum_{k=1}^K x^{(k)}$  and  $\langle x^2 \rangle = \frac{1}{K} \sum_{k=1}^K (x^{(k)})^2$ . They approach the correct values 0 and 1 as  $K$  increases

For the standard Gaussian the true values are:

$$\langle x \rangle = 0 \quad \langle x^2 \rangle = 1$$

Running averages converge to correct values as  $K \rightarrow \infty$ .

# Wrong Implementation — What Breaks It

What if  $\Delta x \sim \text{Uniform}(-\frac{1}{2}, +1)$  instead of  $\text{Uniform}(-c, +c)$ ?

**What breaks:**

- Transition  $0 \rightarrow 1$  is possible ( $\Delta x = 1$  is in range)
- Transition  $1 \rightarrow 0$  requires  $\Delta x = -1$ , which is **outside**  $(-\frac{1}{2}, +1)$
- So  $T(0 \rightarrow 1) \neq T(1 \rightarrow 0)$  — **asymmetric proposal**

This violates **Detailed Balance**:

$$P(x) T(x \rightarrow x') = P(x') T(x' \rightarrow x)$$

# Why Symmetry is Required

**Detailed balance** requires a symmetric proposal:

$$T(x \rightarrow x') = T(x' \rightarrow x)$$

For  $\Delta x \sim \text{Uniform}(-c, +c)$ :

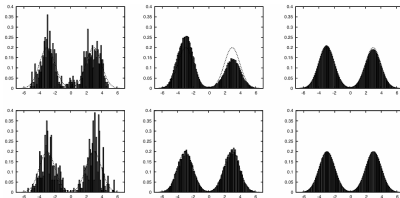
- Probability of proposing  $x \rightarrow x'$  is  $\frac{1}{2c}$
- Probability of proposing  $x' \rightarrow x$  is also  $\frac{1}{2c}$  ✓

**The fix is simple:** always use symmetric interval  $(-c, +c)$

# Beyond Gaussian — Bimodal Distribution (§4.4)

**Target:** superposition of two Gaussians

$$P(x) = \frac{e^{-(x-3)^2/2} + e^{-(x+3)^2/2}}{2\sqrt{2\pi}}$$



**Fig. 4.9** The histograms of  $x^{(1)}, x^{(2)}, \dots, x^{(K)}$  with  $K = 10^3, 10^5, 10^7$ . The dashed lines are the target distribution  $P(x) = \frac{e^{-(x-3)^2/2} + e^{-(x+3)^2/2}}{2\sqrt{2\pi}}$ . The step size is 0.5 (top) or 5.0 (bottom). When the step size is 0.5, the deviation from the target distribution is visible even with  $10^7$  configurations. When the step size is 5.0, the convergence to the target distribution is much faster

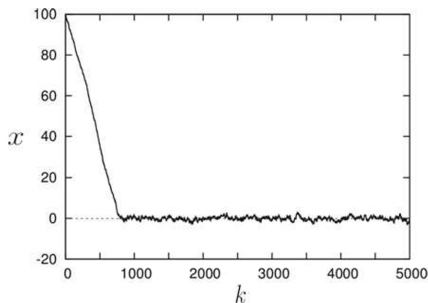
- Two peaks appear at  $x = \pm 3$  — correctly captured
- **Only the action formula  $S(x)$  changed**
- Proposal, acceptance test, chain structure — all identical

This shows how **general** the Metropolis algorithm is

## Burn-In (Thermalization) — Fig. 4.4

**Problem:** Starting far from the typical region corrupts early samples

**Fig. 4.4** The history of the simulation of the Gaussian distribution via the Metropolis algorithm. The step size is  $c = 0.5$ . Because we chose the initial value to be  $x = 100$ , which is very far from the typical values, the strong correlation with the initial value remains for a while, and it took a lot of steps to reach the typical values  $|x| \sim 1$



- Example:  $x^{(0)} = 100$ , typical values  $|x| \sim 1$
- Chain takes  $\sim 800$  steps to reach typical region
- Early samples are **biased** by the initial condition
- **Solution:** discard all samples with  $k \leq K_{\text{cut}}$

# How to Choose $K_{\text{cut}}$

**Method:** plot  $\langle x^2 \rangle$  as a function of  $K_{\text{cut}}$

- At small  $K_{\text{cut}}$ : value changes as we vary the cutoff — initial condition still has influence
- At sufficiently large  $K_{\text{cut}}$ : value **stops changing** — initial condition's effect has gone
- That is the safe discard point

**Physical analogy — ice in water:**

- Until ice melts: heat flows from water to ice (non-thermalized)
- After ice melts: temperature uniform everywhere (thermalized)
- The chain behaves the same way

In Fig. 4.4, thermalization is achieved by  $k \approx 800$ . Discarding  $k \leq 1000$  is safe.

# Meaning of Thermalization

## First meaning

Chain has escaped the influence of initial condition  $x^{(0)}$

## Second meaning

Enough **independent** samples collected so that running averages stabilize

### Why they are different:

- After burn-in, samples are still **autocorrelated**
- With  $K$  samples and autocorrelation length  $\tau$ :

$$\text{effective independent count} \approx \frac{K}{\tau}$$

- If  $K/\tau$  is small,  $\langle x^2 \rangle$  still fluctuates significantly as we add more samples

## Thermalized (second sense)

$\langle x^2 \rangle$  barely changes as more samples are added



# What is Autocorrelation?

In MCMC, consecutive samples are **not independent**:

$$x^{(k+1)} \text{ depends on } x^{(k)}$$

**Why this matters:**

- Standard error formula assumes independent samples
- If samples are correlated, statistical error is **underestimated**
- Two samples are independent only if separated by  $\geq \tau$  steps

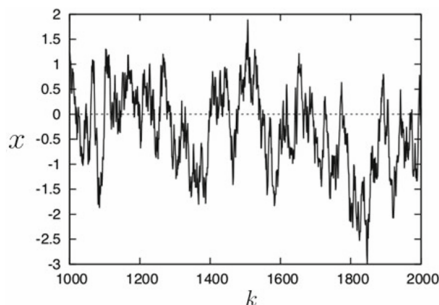
**Autocorrelation Function:**

$$\rho(\tau) = \frac{\langle x^{(k)} x^{(k+\tau)} \rangle - \langle x \rangle^2}{\langle x^2 \rangle - \langle x \rangle^2}$$

- $\rho(0) = 1$  always
- $\rho(\tau) \rightarrow 0$  as  $\tau \rightarrow \infty$  for a good chain
- **Autocorrelation length** = lag  $\tau$  where  $\rho(\tau) \approx 0$

# Autocorrelation in Practice — Fig. 4.5

**Fig. 4.5** The zoom-in of Fig. 4.4, from  $k = 1000$  to  $k = 2000$ . Strong correlations survive at least for 20 or 30 steps



- Zoom into  $k = 1000$  to  $k = 2000$  (well after burn-in)
- Strong correlations survive for **at least 20–30 steps**
- From  $K = 1000$  samples with  $\tau = 25$ : only  $\approx 40$  truly independent samples

## Key consequence

Statistical errors are much larger than naively assumed — we must measure  $\tau$  quantitatively

# Jackknife Method — What It Does

**Goal:** Estimate autocorrelation length  $\tau$  and true statistical error

**Idea:** divide samples into groups of width  $w$

- Group  $l$  contains samples  $x^{((l-1)w+1)}, \dots, x^{(lw)}$
- Compute the group average:

$$\tilde{f}^{(l,w)} = \frac{1}{w} \sum_{j=(l-1)w+1}^{lw} f(x^{(j)})$$

- Compute the Jackknife error:

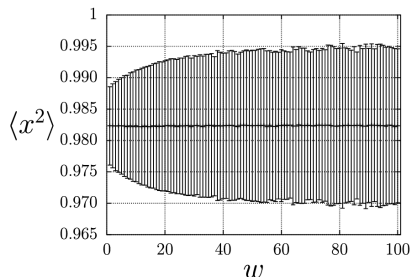
$$\Delta_w = \sqrt{\frac{1}{n(n-1)} \sum_{l=1}^n \left( \tilde{f}^{(l,w)} - \bar{f} \right)^2}$$

**What happens as  $w$  increases:**

- $w < \tau$ : groups still correlated  $\Rightarrow \Delta_w$  keeps **growing**
- $w \approx \tau$ : groups become independent  $\Rightarrow \Delta_w$  **plateaus**
- $w > \tau$ :  $\Delta_w$  stays roughly constant

# Jackknife — Reading the Plot — Fig. 4.6

**Fig. 4.6** The expectation value of  $x^2$  and the Jackknife error



**Reading off results:**

$$w_c = \text{autocorrelation length} = \tau$$

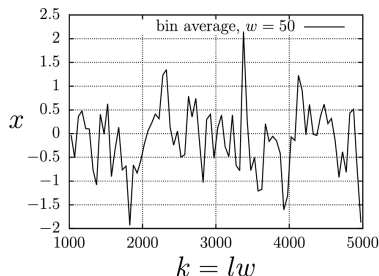
$$\Delta_{w_c} = \text{true statistical error}$$

## Key point

Using  $\Delta_1$  (no grouping) as your error **underestimates** the true error by a factor of  $\sqrt{w_c}$

# Jackknife — Final Result — Fig. 4.7

**Fig. 4.7** The average of each group  $\bar{f}^{(l,w)}$ , with the width  $w = 50$



- With  $w = 50$  (safely above  $w_c \approx 40$ ): groups look independent in Fig. 4.7
- Final estimate:

$$\boxed{\langle x^2 \rangle = 0.982 \pm 0.012} \quad (\text{true value} = 1 \checkmark)$$

This is how MCMC results are correctly reported with proper error bars

# Step Size $c$ — What It Controls

Recall:  $x \sim \text{Uniform}(-c, +c)$ ,  $c$  is a **free parameter we choose**

| Step size                     | Acceptance rate | Chain behaviour              |
|-------------------------------|-----------------|------------------------------|
| Very small $c$                | $\approx 100\%$ | Tiny steps, very slow mixing |
| <b>Optimal <math>c</math></b> | <b>30%–80%</b>  | <b>Efficient</b>             |
| Very large $c$                | $\approx 0\%$   | Almost all rejected, stuck   |

## Both extremes are bad

Small  $c$ : chain barely moves despite high acceptance

Large  $c$ : almost all proposals rejected

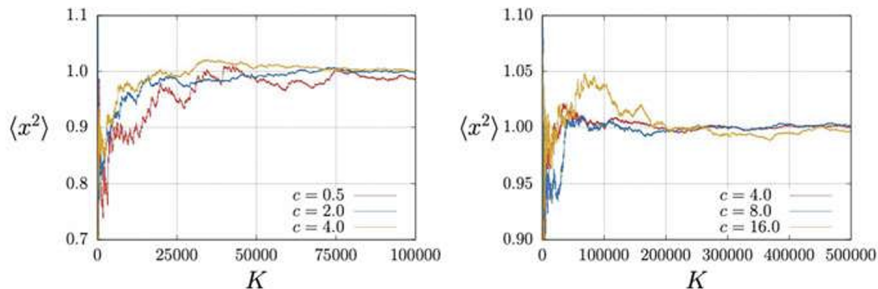
Either way: autocorrelation  $\tau$  is large, simulation is inefficient

## Step Size Trade-off — Table 4.1

| Step size $c$ | Acceptance rate | $c \times \text{A.R.}$ |
|---------------|-----------------|------------------------|
| 0.5           | 0.9077          | 0.454                  |
| 1.0           | 0.8098          | 0.810                  |
| 2.0           | 0.6281          | 1.256                  |
| 3.0           | 0.4864          | 1.459                  |
| 4.0           | 0.3911          | 1.564                  |
| 6.0           | 0.2643          | 1.586                  |
| 8.0           | 0.1993          | 1.594                  |

- $c = 0.5, 1.0$ : acceptance high  $\Rightarrow c$  **too small**
- $c > 4$ :  $c \times \text{A.R.}$  nearly constant  $\Rightarrow c$  **too large**
- **Sweet spot**:  $c = 2.0$  to  $4.0$  (A.R.  $\approx 30\%$ – $80\%$ )

# Convergence for Different Step Sizes — Fig. 4.8



**Fig. 4.8**  $\langle x^2 \rangle = \frac{1}{K} \sum_{k=1}^K (x^{(k)})^2$  with several different choices of the step size  $c$ . The simulation is not efficient when the step size is too big or too small

- $c = 2.0$  and  $c = 4.0$ : **fastest convergence**
- $c = 0.5$ : converges slowly (tiny steps)
- $c = 8.0$ : converges slowly (mostly rejected)



# Random Walk Argument — Why $\tau \propto 1/c^2$

The Metropolis proposal is a **random walk** with step size  $c$ . In a random walk, after  $n$  steps:

$$\text{change in } x \approx c\sqrt{n}$$

To explore a region of size  $\sim 1$  (standard Gaussian):

$$c\sqrt{n} \sim 1 \implies n \sim \frac{1}{c^2}$$

Therefore:

$$\tau \propto \frac{1}{c^2}$$

## Conclusion

Larger  $c \Rightarrow$  shorter  $\tau$  — but *only* until acceptance rate drops too low  
This is why there is an optimal  $c$

# Sweet Spot for Acceptance Rate

From the random walk argument and Table 4.1:

$$\tau \propto \frac{1}{c^2} \quad \text{but also} \quad \text{large } c \Rightarrow \text{low A.R.} \Rightarrow \text{chain stuck}$$

## Optimal strategy:

- Tune  $c$  so that acceptance rate is between **30% and 80%**
- This minimises  $\tau$ , giving most independent samples per iteration
- For the Gaussian:  $c = 2.0$  to  $c = 4.0$  is optimal

## Practical Rule

Run a short test simulation and measure acceptance rate

A.R.  $> 80\% \Rightarrow$  increase  $c$

A.R.  $< 30\% \Rightarrow$  decrease  $c$

## Box-Muller — Zero Autocorrelation (§4.3.5)

Box-Muller generates each sample **independently** of the previous:

$$T(x \rightarrow x') = P(x')$$

**As a special case of MCMC:**

- Satisfies all 4 MCMC conditions trivially
- Detailed balance:

$$P(x) \cdot T(x \rightarrow x') = P(x) \cdot P(x') = P(x') \cdot T(x' \rightarrow x) \quad \checkmark$$

- Autocorrelation length  $\tau = 1$  — the ideal extreme

|                 | Box-Muller    | Metropolis        |
|-----------------|---------------|-------------------|
| Autocorrelation | <b>Zero</b>   | $\tau > 0$        |
| Works for       | Gaussian only | <b>Any</b> $P(x)$ |
| Needs $Z$       | Yes           | <b>No</b>         |

Box-Muller = ideal MCMC case. Metropolis trades autocorrelation for **full generality**.

# Summary

## What we covered

- 1 **Algorithm:** Metropolis samples from  $P(x) = e^{-S(x)}/Z$  without needing  $Z$
- 2 **Convergence:** Histogram and expectation values converge to correct targets as  $K \rightarrow \infty$
- 3 **Diagnostics:** Burn-in removes initial bias; Jackknife measures autocorrelation length  $\tau$
- 4 **Efficiency:** Tune  $c$  for 30%–80% acceptance to minimise  $\tau \propto 1/c^2$

## Key Insight

$Z$  cancels in the acceptance ratio — this is **why Metropolis works** for distributions we cannot normalise

# Thank You

Questions?

- Reference:**
1. M. Hanada & S. Matsuura  
*MCMC from Scratch*  
Springer Nature- 2022;
  2. Metropolis et al. (1953)